

Day 39–46

GitHub Actions & CI/CD — Beginner-Friendly Practice Notes

Goal: Understand CI/CD first, then build GitHub Actions workflows step-by-step, work with runners, jobs, secrets, artifacts, Docker publishing, and reusable automation.

How to use this PDF: Read the concept → type the example yourself → push to GitHub → inspect the run → intentionally break something → fix it → write what you learned.

Source basis: These notes are constructed from the supplied Day 39–46 challenge file. The task order, expected outputs, terminology, hints, and practice objectives are preserved.

Day 39 — What is CI/CD?

What you learn

- Why CI/CD exists and what problem it solves.
- The difference between Continuous Integration, Continuous Delivery, and Continuous Deployment.
- The basic anatomy of a pipeline.
- How to read an existing GitHub Actions workflow.

Beginner idea

- Without automation, developers may manually build, test, and deploy. This becomes risky as a team grows.
- CI/CD is a practice, not a single tool. GitHub Actions, Jenkins, GitLab CI and CircleCI are tools that implement CI/CD.
- A failed pipeline is useful: it catches a problem before the change moves further.

CI vs CD

- **Continuous Integration (CI):** Developers frequently integrate changes. Automated checks such as tests/builds catch problems early.
- **Continuous Delivery:** Code is automatically prepared and kept ready for release, but production release can still require a decision/approval.
- **Continuous Deployment:** A successful pipeline automatically deploys changes to production.

Pipeline anatomy

- **Trigger:** What starts the pipeline, such as a push or pull request.
- **Stage:** A logical phase such as build, test, or deploy.
- **Job:** A unit of work inside a stage.
- **Step:** One command or action inside a job.
- **Runner:** The machine that executes a job.
- **Artifact:** Output produced by a job, such as a report or build file.

Practice

- Think about five developers manually deploying to production. List what can go wrong and explain “it works on my machine.”
- Draw: Developer push → Test → Docker Build → Staging Deploy.
- Explore an open-source GitHub repository. Find `.github/workflows/`, open one workflow, and note its trigger, number of jobs, and purpose.

Expected output

- `day-39-cicd-concepts.md`
- A pipeline diagram (hand-drawn or text-based).

Day 40 — Your First GitHub Actions Workflow

What you learn

- Create your first workflow.
- Understand `on:`, `jobs:`, `runs-on:`, `steps:`, `uses:`, `run:`, and step name.
- Read a successful and failed workflow run.

Setup

- Create a public repository named `github-actions-practice`.
- Clone it locally.
- Create `.github/workflows/`.

Starter workflow

- Create `.github/workflows/hello.yml`. It triggers on every push, has one job named `greet`, runs on `ubuntu-latest`, checks out the code, and prints a greeting.

Practice YAML

Example

```
- name: Show branch
  run: echo "Branch: ${ github.ref_name }"

- name: List files
  run: ls -la

- name: Runner OS
  run: uname -a

- name: Current date
  run: date
```

Break it on purpose

- Add a step that fails, for example with `exit 1`. Push it and inspect the red run. Fix it and push again.

Extra steps

- Print current date/time.
- Print the triggering branch using GitHub's branch context.
- List repository files.
- Print the runner operating system.

Expected output

- `hello.yml`
- `day-40-first-workflow.md`
- Screenshot of a green pipeline run.

Day 41 — Triggers & Matrix Builds

Triggers

- `<b>push:</b>` run after code is pushed.
- `<b>pull_request:</b>` run for pull request activity.
- `<b>schedule:</b>` run using cron.
- `<b>workflow_dispatch:</b>` manually run a workflow and optionally collect inputs.

Pull request practice

- Create `pr-check.yml`. Trigger only when a PR is opened/updated against main. Print the branch name.

Schedule practice

- Add a scheduled trigger for every day at midnight UTC: `<b>0 0 * * *</b>`.
- Challenge question: every Monday at 9 AM UTC is commonly represented as `<b>0 9 * * 1</b>`.

Manual workflow

- Create `manual.yml` with `workflow_dispatch` and an environment input such as staging/production. Print the selected value.

Matrix builds

- A matrix runs the same job for multiple combinations.
- Start with Python 3.10, 3.11, 3.12.
- Then add two operating systems. With 3 Python versions × 2 OS values, the matrix produces 6 combinations before exclusions.

Exclude & fail-fast

- Exclude one combination such as Python 3.10 on Windows.
- `<b>fail-fast: true</b>` (default) can cancel in-progress matrix jobs when a matrix job fails.
- `<b>fail-fast: false</b>` allows the other matrix jobs to continue.

Practice

- Watch the matrix jobs run in parallel. Add an exclusion. Deliberately fail one job and compare fail-fast true vs false.

Expected output

- `pr-check.yml`, `manual.yml`, `matrix.yml` and/or updated workflows.
- `day-41-triggers.md`.
- Screenshots of runs.

Useful YAML patterns

```
# Pull request
on:
  pull_request:
    branches: [main]

# Schedule
on:
  schedule:
```

```
- cron: '0 0 * * *'

# Manual
on:
  workflow_dispatch:
    inputs:
      environment:
        description: Environment
        required: true

# Matrix
strategy:
  fail-fast: false
  matrix:
    python-version: ['3.10', '3.11', '3.12']
    os: [ubuntu-latest, windows-latest]
    exclude:
      - os: windows-latest
        python-version: '3.10'
```

Day 42 — Runners: GitHub-Hosted & Self-Hosted

What is a runner?

- Every job needs a machine. A runner is the machine that executes the job.
- **GitHub-hosted:** GitHub provides/manages the runner environment.
- **Self-hosted:** you provide and manage the machine. It can be a local machine or cloud VM.

GitHub-hosted practice

- Create 3 jobs using ubuntu-latest, windows-latest and macos-latest. In each, print OS name, hostname and current user.

Pre-installed tools

- On ubuntu-latest, print Docker, Python, Node and Git versions. Knowing what is already installed can simplify setup and reduce repeated installation work.

Self-hosted runner

- Repository → Settings → Actions → Runners → New self-hosted runner. Choose Linux and follow GitHub's generated instructions on your machine or VM.
- Start the runner and verify it shows Idle with a green dot.

Use your own machine

- Create self-hosted.yml with runs-on: self-hosted. Print hostname and working directory. Create a file and verify it exists on the machine after the job.

Labels

- Add a label such as my-linux-runner and target it with runs-on: [self-hosted, my-linux-runner]. Labels help select a particular runner when multiple runners exist.

Comparison

	GitHub-hosted	Self-hosted
Who manages it?	GitHub	You / your organization
Cost	Uses GitHub-hosted minutes/plan limits	Your machine/cloud infrastructure cost
Pre-installed tools	Runner image provides tools	You control/install tools
Good for	General CI/CD	Special hardware, private network, custom environments
Security concern	Less machine management for you	You must secure and maintain the runner

Day 43 — Jobs, Steps, Env Vars & Conditionals

Multi-job workflow

- Create build, test and deploy jobs. Use needs so test waits for build and deploy waits for test.
- Dependency chain: build → test → deploy.

Environment variables

- Workflow-level: APP_NAME: myapp.
- Job-level: ENVIRONMENT: staging.
- Step-level: VERSION: 1.0.0.
- Print them in one step. Also print the commit SHA and actor using GitHub context.

Job outputs

- One job creates an output such as today's date. Another job reads it with needs.<job>.outputs.<name>.
- The challenge hint uses: echo "date=\$(date)" >> \$GITHUB_OUTPUT.

Conditionals

- Run a step only on main.
- Run a step only when the previous step failed.
- Run a job only on push events, not pull requests.
- continue-on-error: true lets a step fail without making the overall job fail in the normal way.

Smart pipeline

- Trigger on push to any branch. Run lint and test in parallel. Then run a summary job after both. The summary identifies main vs feature branch and prints the commit message.

Practice

- First make jobs independent. Then add needs. Add variables at each scope. Pass an output between jobs. Deliberately fail a step and test the failure condition.

Expected output

- multi-job.yml, smart-pipeline.yml and other practice workflows.
- day-43-jobs-steps.md.

Useful YAML patterns

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Building the app"

  test:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Running tests"

  deploy:
    needs: test
```

```
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying"

# Job output idea
# echo "date=$(date)" >> $GITHUB_OUTPUT
# Read with:
# ${ needs.job-name.outputs.date }
```

Day 44 — Secrets, Artifacts & Running Real Tests in CI

Secrets

- Create repository secret MY_SECRET_MESSAGE in Settings → Secrets and Variables → Actions.
- Use the secret without hardcoding it. Never print the actual secret value in logs.
- GitHub masks secret values in logs automatically, but you should still design workflows so secrets are not exposed.

Secret as environment variable

- Pass a secret into a step through an environment variable. Also prepare DOCKER_USERNAME and DOCKER_TOKEN secrets for Day 45.

Artifacts

- An artifact is a file saved from a workflow run, such as a test report or log.
- Use actions/upload-artifact@v4 to upload it.
- Use actions/download-artifact@v4 in another job to retrieve it.

Real tests

- Put a Python or Shell script in the repository. Checkout code, install dependencies, run the script, and let a non-zero exit code fail the pipeline.
- Break the script intentionally, observe red, fix it, and observe green.

Caching

- Use actions/cache@v4 when installing dependencies. Run the workflow twice and compare time. Think about what data is cached and where the cache is reused.

Practice

- Create a test-report.txt or log file and upload it. Download it from the Actions tab. Pass an artifact between two jobs. Add a real script and test both success and failure.

Expected output

- day-44-secrets-artifacts.md
- Artifact download screenshot.
- Passing CI test screenshot.

Secrets, artifacts and cache — quick patterns

```
# Secret as environment variable
env:
  MY_MESSAGE: ${ secrets.MY_SECRET_MESSAGE }
run: |
  if [ -n "$MY_MESSAGE" ]; then
    echo "The secret is set: true"
  fi

# Upload
- uses: actions/upload-artifact@v4
  with:
    name: test-report
    path: test-report.txt

# Download in another job
```

```
- uses: actions/download-artifact@v4
  with:
    name: test-report
```

Day 45 — Docker Build & Push in GitHub Actions

Goal

- A push to GitHub automatically builds a Docker image and publishes it to Docker Hub. This connects source control, CI and container distribution.

Prepare

- Use the app from Day 36 or another simple Dockerfile. Add it to github-actions-practice. Ensure DOCKER_USERNAME and DOCKER_TOKEN secrets exist.

Pipeline journey

- Git push → GitHub Actions trigger → checkout → Docker build → tag → Docker Hub login → push → pull → run container.

Tags

- Publish a latest tag and a commit-based tag such as sha-<short-commit-hash>. The commit tag makes a specific build easier to identify.

Only push on main

- Feature branches can build the image but the push step should run only for main. This lets you validate builds without publishing every branch's image.

Practice

- Build on push to main. Verify logs. Log in with secrets. Push both tags. Push a feature branch and verify it builds but does not publish. Pull and run the image locally or on a cloud server.

Important actions from the supplied task

- docker/login-action@v3
- docker/build-push-action@v5

Expected output

- docker-publish.yml
- Docker Hub image.
- README status badge.
- day-45-docker-cicd.md and pipeline screenshot.

Docker publishing workflow — simplified learning version

```
name: Docker Publish

on:
  push:
    branches: [main]

jobs:
  docker:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: docker/login-action@v3
        with:
```

```
    username: ${ secrets.DOCKER_USERNAME }
    password: ${ secrets.DOCKER_TOKEN }

- uses: docker/build-push-action@v5
  with:
    push: true
    tags: yourusername/yourrepo:latest
```

Day 46 — Reusable Workflows & Composite Actions

Why reuse?

- Real teams avoid copying the same workflow logic into every repository. Reusable workflows and composite actions help standardize automation.

Reusable workflow

- A reusable workflow is a workflow designed to be called by another workflow.
- `workflow_call` is the trigger used for this purpose.
- It lives under `.github/workflows/`.
- A caller invokes it with a job-level `uses` reference, unlike a normal action used inside a step.

Build reusable workflow

- Create `reusable-build.yml` with `workflow_call`.
- Inputs: `app_name` (string, required) and `environment` (string, required/default staging as specified by the task).
- Secret: `docker_token` (required).
- Job checks out code, prints the app/environment and confirms the token is set without printing it.

Caller workflow

- Create `call-build.yml`. Trigger on push to main. Call `./github/workflows/reusable-build.yml` and pass inputs and the Docker token secret.

Outputs

- Expose `build_version` from the reusable workflow. Generate a version such as `v1.0-<short-sha>`. A second caller job waits with needs and prints the returned version.

Composite action

- Create `.github/actions/setup-and-greet/action.yml`.
- Inputs: `name` and `language` (default `en`).
- Use `runs: using: "composite"`. The action prints a greeting, current date, runner OS, and an output `greeted=true`.
- Use it from a workflow with `uses: ./github/actions/setup-and-greet`.

Comparison

	Reusable Workflow	Composite Action
Triggered by	<code>workflow_call</code>	<code>uses:</code> in a step
Can contain jobs?	Yes	No; it is used within a job step sequence
Can contain multiple steps?	Yes	Yes
Lives where?	<code>.github/workflows/</code>	Usually <code>.github/actions/<action>/action.yml</code>
Can accept secrets directly?	Yes, via <code>workflow_call</code> secrets	Pass values as inputs/environment as appropriate
Best for	Reusable multi-job pipeline logic	Reusable step-level logic

Reusable workflow — simplified structure

```
# .github/workflows/reusable-build.yml
on:
  workflow_call:
    inputs:
      app_name:
        required: true
        type: string
      environment:
        required: true
        type: string
        default: staging
    secrets:
      docker_token:
        required: true

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "Building ${ inputs.app_name } for ${ inputs.environment }"
```

Caller idea

```
# .github/workflows/call-build.yml
on:
  push:
    branches: [main]

jobs:
  build:
    uses: ../.github/workflows/reusable-build.yml
    with:
      app_name: "my-web-app"
      environment: "production"
    secrets:
      docker_token: ${ secrets.DOCKER_TOKEN }
```

Composite action structure

```
# .github/actions/setup-and-greet/action.yml
name: Setup and Greet
inputs:
  name:
    required: true
  language:
    default: en

runs:
  using: composite
  steps:
    - shell: bash
      run: echo "Hello ${ inputs.name }"
```

Final Revision — Days 39–46

Think of the eight days as one journey:

```
Day 39 → Understand CI/CD
↓
Day 40 → First GitHub Actions workflow
↓
Day 41 → Triggers + matrix
↓
Day 42 → Runners
↓
Day 43 → Jobs + variables + conditions
↓
Day 44 → Secrets + artifacts + real tests
↓
Day 45 → Docker build + Docker Hub
↓
Day 46 → Reusable workflows + composite actions
```

Beginner self-check

- Can I explain CI, Continuous Delivery and Continuous Deployment in my own words?
- Can I find a workflow under `.github/workflows/`?
- Do I understand trigger, job, step and runner?
- Can I create a workflow that runs on push?
- Can I use a pull request, schedule and manual trigger?
- Can I explain why a matrix creates multiple jobs?
- Can I tell GitHub-hosted and self-hosted runners apart?
- Can I use needs to create a job dependency?
- Can I use environment variables and GitHub context values?
- Can I pass an output from one job to another?
- Can I store and use secrets without printing them?
- Can I upload/download an artifact?
- Can I make a failing test turn the pipeline red?
- Can I build and publish a Docker image from GitHub Actions?
- Can I explain reusable workflow vs composite action?

Practice Rule

Do not only copy YAML. Type it, run it, inspect every step, break it deliberately, read the error, fix it, and write one sentence explaining why it worked. That is the fastest way to turn GitHub Actions syntax into a practical skill.

Suggested repository structure

```
github-actions-practice/
├── .github/
│   └── workflows/
│       ├── hello.yml
│       ├── pr-check.yml
│       ├── manual.yml
│       ├── matrix.yml
│       ├── self-hosted.yml
│       ├── multi-job.yml
│       ├── smart-pipeline.yml
│       └── docker-publish.yml
```

```
■ ■ ■ reusable-build.yml
■ ■ ■ call-build.yml
■ ■ actions/
■ ■ setup-and-greet/
■ ■ action.yml
■ Dockerfile
■ README.md
■ your-test-or-app-files
```

Day-wise documentation files

```
day-39-cicd-concepts.md
day-40-first-workflow.md
day-41-triggers.md
day-42-runners.md
day-43-jobs-steps.md
day-44-secrets-artifacts.md
day-45-docker-cicd.md
day-46-reusable-workflows.md
```